

Figure 11: Colouring the Interference Graph (before ILP)

Nodes 12 and 13) due to which the graph is un-colourable, i.e., we do not use this ILP. The second possibility is to keep the ILP, but since the graph is un-colourable (we do not have enough physical registers), we would need to add spill-code. (For details on register spilling, please see ‘Dissecting Register Allocation’ in the June 2010 issue of *LINUX For You*.)

The best option?

We have explored three methods of applying register allocation and ILP. We have seen how the positioning of the register allocation module can increase or decrease the scope of ILP. Now, the question is: of the three methods, which should we choose?

We saw that in Method I, we made optimal use of available registers, but created unnecessary dependencies due to register re-use—thereby reducing the scope of ILP. So, in case the number of processor registers is limited, Method I would be suitable.

In Method II, we were able to generate an efficient schedule since unnecessary dependencies were not created. We made liberal use of registers to achieve ILP. However, the increased use of registers could result in register spilling, if the number of registers on the processor is limited. Therefore, if the processor has a good amount of physical registers, Method II is preferable.

Method III aims at lowering the register pressure and increasing flexibility in instruction scheduling. But, as we saw earlier, it might result in an un-colourable interference

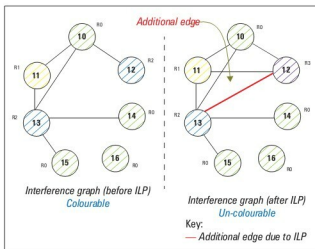


Figure 12: Comparing Interference Graphs

graph, after achieving ILP. In that case, you would need to generate spill-code. This may or may not be a better solution, because addition of spill-code implies the addition of two memory transfer instructions (one memory write, one read). Not only would this be an overhead, but also, spilling means you need to re-construct the interference graph, which in turn increases compilation time. If we are able to combine these additional transfer instructions in parallel with some instruction, then it would be beneficial. Therefore, Method III would provide the best result in case the processor supports a high degree of parallelism.

In forthcoming articles, I will discuss the concept of pipelining, and how it is closely related with ILP. 

References and further reading

1. Deepti Sharma. ‘Dissecting Instruction Level Parallelism’, *LINUX For You*, May 2010
2. Deepti Sharma. ‘Dissecting Register Allocation’, *LINUX For You*, June 2010
3. Brasier, Sweany and Beatty. White paper on ‘CRAIG: A Practical Framework for Combining Instruction Scheduling and Register Assignment’

By: Deepti Sharma

The author is a senior software engineer with considerable experience in the development of compilers and tools for embedded systems. She loves writing for LFY. She can be reached at deepti.acmet@gmail.com

A Computer Magazine that does not talk about Music, Games, Photo albums—**Only Business**

Read Benefit: A Business Minded Computer Magazine



www.benefitmag.com